

录入Skill标准

注：

进入 `.agents/skills/` 只代表可调用，不代表前台展示；前台展示需要额外登记。

关键的原则：`skills-inbox` 管“收集”，`skills` 管“可复用能力”，`registries` 管“系统索引”，`brain-data.json` 管“前台表达”

完整目录详见：[📄 Coding仓库目录树结构说明](#)

一、SKill提交流程标准

提交时统一进入：

```
1 .agents/skills-inbox/<skill-slug>/
```

二、正式Skill标准目录

正式目录树：

1	.agents/skills/<skill-slug>/ kebab-case, 如 relay-to-design-md	= 单个正式 Skill, 目录名使用英文
2		
3	— README.md 是什么、谁维护、适合什么场景、如何使用	= 给人看的说明文档: 这个 Skill
4		
5	— SKILL.md 触发条件、任务边界、执行步骤、读取顺序、输出要求、禁止事项	= 给 Agent 执行的核心说明:
6		
7	— references/ 流程、背景和来源资料, 不放验收样例	= Skill 依赖的知识依据层; 放规则、
8	— overview.md 术语定义	= 背景说明、适用范围、核心概念、
9	— rules.md 禁止事项	= 规则、原则、判断标准、边界条件、
10	— workflow.md 异常处理方式	= 执行流程、阶段门、检查点、
11	— source-docs/ 设计规范、会议纪要等, 用于保留可追溯性	= 可选: 原始来源资料、业务文档、
12		
13	— examples/ 测试、教学和回归验证	= Skill 的可验证样例层; 用于审核、
14	— input-01.md 提供什么材料	= 示例输入: 用户可能怎么提问、
15	— output-01.md 应该产出什么结果、格式和质量标准是什么	= 示例输出: Agent
16		
17	— assets/ 示例素材等静态资源	= 可选: 图片、字体、视频、图标、
18		
19	— scripts/ 批处理脚本	= 可选: 校验、同步、生成、转换、
20		
21	— agents/ 的配置	= 可选: 不同 Agent runtime
22	— openai.yaml 等运行配置	= 可选: OpenAI / Codex

最低提交标准:

1	.agents/skills/<skill-slug>/	
2		
3	— README.md	= 必填：说明用途、场景、输入输出、维护人
4	— SKILL.md	= 必填：说明 Agent 如何执行、何时触发、输出什么、不能做什么
5		
6	— references/	= 必填：至少放一个规则或背景文件
7	└ rules.md	= 必填：核心规则、判断标准、边界条件
8		
9	— examples/	= 必填：至少提供一组输入输出样例
10	└ input-01.md	= 必填：示例输入
11	└ output-01.md	= 必填：标准输出

可选项：

```

1  assets/
2  scripts/
3  agents/openai.yaml

```

一句话版规则：

1 README.md 给人看，SKILL.md 给 Agent 执行，references/ 放知识依据，examples/ 放验收样例；assets/、scripts/、agents/ 都是增强项，不是最低必填。

三、README 参考

正式 Skill 的 **README.md** 标准（建议必填，里面有前台需要拉取的字段）：

```
▼ md
1  # <Skill 名称>
2
3  ## 基础信息
4
5  - 提交人:
6  - 维护人:
7  - 来源:
8  - 所属业务域: 通用 / 品牌域 / 营销域 / 互动域 / 多媒体内容域
9  - 当前状态: 预研 / 进行中 / 已应用+版本号
10
11 ## 用途
12
13 这个 Skill 解决什么问题?
14
15 ## 适用场景
16
17 什么时候应该使用这个 Skill?
18
19 ## 输入
20
21 用户需要提供什么?
22
23 ## 输出
24
25 Skill 会产出什么?
26
27 ## 依赖资料
28
29 引用了哪些文档、设计稿、工具、代码仓库或业务资料?
30
31 ## 边界和风险
32
33 哪些事情不能做? 哪些内容必须人工审核?
34
35 ## 可选补充
36
37 - 量化收益:
38 - 案例链接:
39 - 体验链接:
40 - 预览图:
```

四、常见问题

1、很多同学的Skill:

```
▼  
1  README.md    # 来源、提交人、用途、输入输出、适用场景  
2  SKILL.md     # Skill 主体
```

这只能算 **最低可运行标准**，不能直接算“完整正式标准”

如果 Skill 依赖复杂规则、业务资料、模板、示例、脚本，就必须升级到完整结构

```
▼  
1  .agents/skills/<skill-slug>/  
2  |  
3  |— README.md  
4  |— SKILL.md  
5  |— references/  
6  |— examples/  
7  |— assets/  
8  |— scripts/  
9  |— agents/
```

三个档:

```
▼  
1  L0 最低可运行:  
2  README.md  
3  SKILL.md  
4  
5  L1 推荐标准:  
6  README.md  
7  SKILL.md  
8  references/  
9  examples/  
10  
11 L2 完整标准:  
12 README.md  
13 SKILL.md  
14 references/  
15 examples/  
16 assets/  
17 scripts/  
18 agents/
```

2、有同学将skill.md写成了<skill-name>.md，不建议这样。（文件名用标准，标题可以个性化。）

因为

```
1 .agents/skills/<skill-slug>/SKILL.md
```

这个路径是给 Agent runtime 识别的“入口契约”。很多工具不会去猜 `relay-to-design-md.md`、`design-review.md` 这种名字，它只会找固定的 `SKILL.md`。一旦改名，这个 Skill 对人还可读，但对 Agent 可能就“失联”了。

所以

```
1 必须保留：
2  SKILL.md
3
4 可以额外增加：
5  README.md
6  references/<skill-name>.md
7  references/overview.md
```

比如：

```
1 .agents/skills/design-review/
2 |
3 |— README.md
4 |— SKILL.md           = 固定入口，不能改名
5 |— references/
6   |— overview.md
7   |— design-review-rules.md
```

如果大家觉得 `SKILL.md` 太抽象，可以在 `README.md` 里写清楚标题，或者在 `SKILL.md` 第一行写：

```
1 # Design Review Skill
```